COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	**CPE 010 - DATA STRUCTURES AND ALGORITHMS**	**SECTION:**	CPE21S4
DATE PERFORMED:	9 / 16 / 26	**DATE SUBMITTED:**	9 / 17 / 26
NAME:	Mangcucang, Rhianne L.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Activity NO. 8.1 **Sorting Algorithms**

1. PROCEDURE OUTPUT

What procedures did you perform?

Code + Console Output

```cpp
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include "SortingAlgorithms.h"

void printArray(const std::vector<int>& arr) {
    for (size_t i = 0; i < arr.size(); i++) {
        std::cout << arr[i] << " ";
        if ((i + 1) % 10 == 0) std::cout << "\n";
    }
    std::cout << "\n";
}

int main() {
    std::srand(std::time(0));
    std::vector<int> originalArray(100);

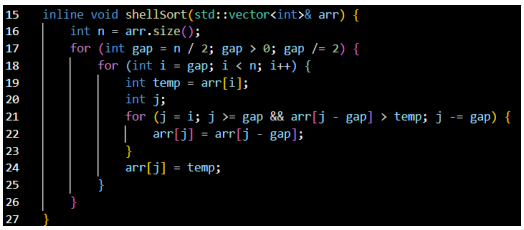
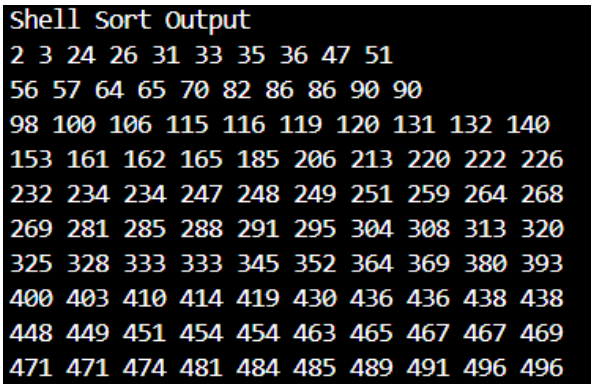
    for (int i = 0; i < 100; i++) {
        originalArray[i] = std::rand() % 500 + 1;
    }
```

```
Initial Random Array (100 elements)
419 469 222 449 162 285 185 64 281 436
220 98 380 47 234 86 436 106 308 65
454 484 313 206 369 448 304 165 2 481
320 288 403 131 259 35 496 36 474 430
249 438 248 465 247 345 393 496 471 471
400 82 333 90 132 463 226 70 120 491
467 251 454 489 451 438 161 264 86 51
57 116 268 213 414 232 485 153 467 295
90 328 410 333 325 140 33 352 24 269
26 234 119 100 3 364 291 56 115 31
```

Observation

The preparation task generates an array composed of 100 random integers. The dataset is completely unsorted and displays a wide variation of values. This provides a clean baseline to accurately verify the sorting execution of Shell Sort, Merge Sort, and

	Quick Sort.

Code + Console Output	 
Observation	**Shell Sort improves upon standard insertion sort by breaking the array into sub-lists using decreasing gap sizes ($N/2, N/4, \dots, 1$). By sorting elements that are far apart first, large value shifts occur early in the pass. As a result, the final insertion sort pass ($gap = 1$) requires significantly fewer moves to fully arrange the 100 elements in ascending order.**

Code + Console Output

```
inline void merge(std::vector<int>& arr, int left, int middle, int right) {
    int n1 = middle - left + 1;
    int n2 = right - middle;

    std::vector<int> L(n1), R(n2);

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[middle + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }
```

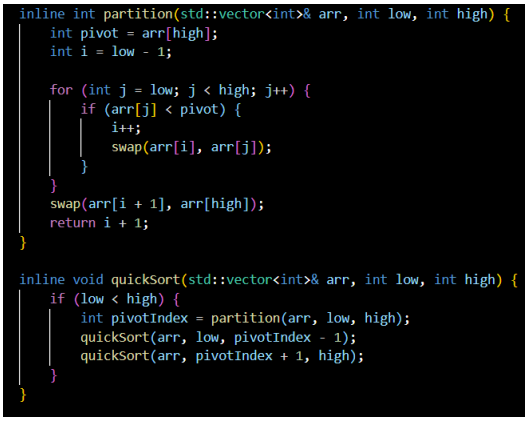
```
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

inline void mergeSort(std::vector<int>& arr, int left, int right) {
    if (left < right) {
        int middle = left + (right - left) / 2;
        mergeSort(arr, left, middle);
        mergeSort(arr, middle + 1, right);
        merge(arr, left, middle, right);
    }
}
```

```
Merge Sort Output
2 3 24 26 31 33 35 36 47 51
56 57 64 65 70 82 86 86 90 90
98 100 106 115 116 119 120 131 132 140
153 161 162 165 185 206 213 220 222 226
232 234 234 247 248 249 251 259 264 268
269 281 285 288 291 295 304 308 313 320
325 328 333 333 345 352 364 369 380 393
400 403 410 414 419 430 436 436 438 438
448 449 451 454 454 463 465 467 467 469
471 471 474 481 484 485 489 491 496 496
```

Observation

Merge Sort demonstrates a divide-and-conquer approach. The initial 100-element array is recursively halved until individual single-element arrays are formed. These single elements are then progressively merged back together in sorted order across sub-arrays of increasing length. The algorithm executes smoothly with guaranteed $O(N \log N)$ complexity regardless of the initial random order.

Code + Console Output	 Quick Sort Output 2 3 24 26 31 33 35 36 47 51 56 57 64 65 70 82 86 86 90 90 98 100 106 115 116 119 120 131 132 140 153 161 162 165 185 206 213 220 222 226 232 234 234 247 248 249 251 259 264 268 269 281 285 288 291 295 304 308 313 320 325 328 333 333 345 352 364 369 380 393 400 403 410 414 419 430 436 436 438 438 448 449 451 454 454 463 465 467 467 469 471 471 474 481 484 485 489 491 496 496
Observation	**Quick Sort partitions the dataset by selecting a pivot element (the last element in this implementation). Elements smaller than the pivot are shifted to the left, while larger elements move to the right. Once the pivot reaches its correct final position, the left and right sub-arrays are recursively partitioned in-place until the entire 100-element array is completely sorted.**

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Yes, the left and right sub-lists resulting from the partition step in Quick Sort can be sorted using other sorting algorithms. Once the pivot element is placed in its correct position, the two sub-lists are completely independent, so any algorithm (such as Insertion Sort, Selection Sort, or Bubble Sort) can be applied to them.

In practical implementations, hybrid algorithms like Introsort use Insertion Sort for small sub-lists because

Insertion Sort has lower overhead and runs faster on small datasets.

```cpp
#include <iostream>
#include <vector>

// Partition method from Quick Sort
int partition(std::vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[high]);
    return i + 1;
}

// Sub-list sorting using Insertion Sort instead of recursion
void insertionSortSublist(std::vector<int>& arr, int low, int high) {
    for (int i = low + 1; i <= high; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= low && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

```cpp
void hybridQuickSort(std::vector<int>& arr, int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);

        // Sort left and right sub-lists using Insertion Sort
        insertionSortSublist(arr, low, pivotIndex - 1);
        insertionSortSublist(arr, pivotIndex + 1, high);
    }
```

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

For the given small array of 15 elements, insertion sort or quick sort will deliver the fastest real world execution time. For very small inputs, insertion sort operates with minimal system overhead because it

eliminates recursive function call stacks and avoids additional memory allocation, outperforming standard algorithms in practical execution speed.

Merge sort achieves an O(N log N) time complexity because its divide and conquer strategy repeatedly divides an array into halves, creating a recursion tree with logarithmic depth. Merging the elements back together across all sub arrays at each level requires linear operations, which results in a consistent overall time complexity across best, average, and worst cases. Similarly, quick sort achieves an average time complexity of O(N log N) because partitioning the array around a chosen pivot takes linear comparisons per level, and a balanced pivot splits the dataset in half to produce a logarithmic recursion tree height.

CONCLUSION

In this activity, implementation of intermediate and advanced sorting algorithms including shell sort, merge sort, and quick sort was performed using C++. Shell sort demonstrated how using a gap sequence reduces element displacement early on, allowing the final insertion sort pass to run much faster, while merge sort showed the divide and conquer principle by splitting arrays down to individual elements before recombining them in sorted order with guaranteed O(N log N) performance. Quick sort highlighted the use of pivot elements to partition arrays in place so sub lists can be sorted recursively. Generating a 100 element random array provided an unbiased test environment, and structuring the code with a separate header file allowed for modular and reusable program design. Each sorting algorithm successfully arranged the random numbers in ascending order, confirming the correct implementation of their underlying gap and partitioning strategies. Furthermore, the supplementary activity proved that sub lists generated by quick sort partitioning can be sorted using other algorithms like insertion sort, which forms the core of modern hybrid algorithms such as introsort. It also showed that small datasets perform faster under insertion sort due to low overhead, while merge sort and quick sort achieve O(N log N) time complexity through logarithmic tree splitting combined with linear comparisons at each level.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.